



Using CI/CD with Google Kubernetes Engine

Using CI/CD with Google Kubernetes Engine

- 01 Continuous integration and continuous deployment (CI/CD)
- 02 Constructing a CI/CD pipeline
- 03 CI/CD products
- 04 Best practices for using CI/CD on Google Cloud



Using CI/CD with Google Kubernetes Engine

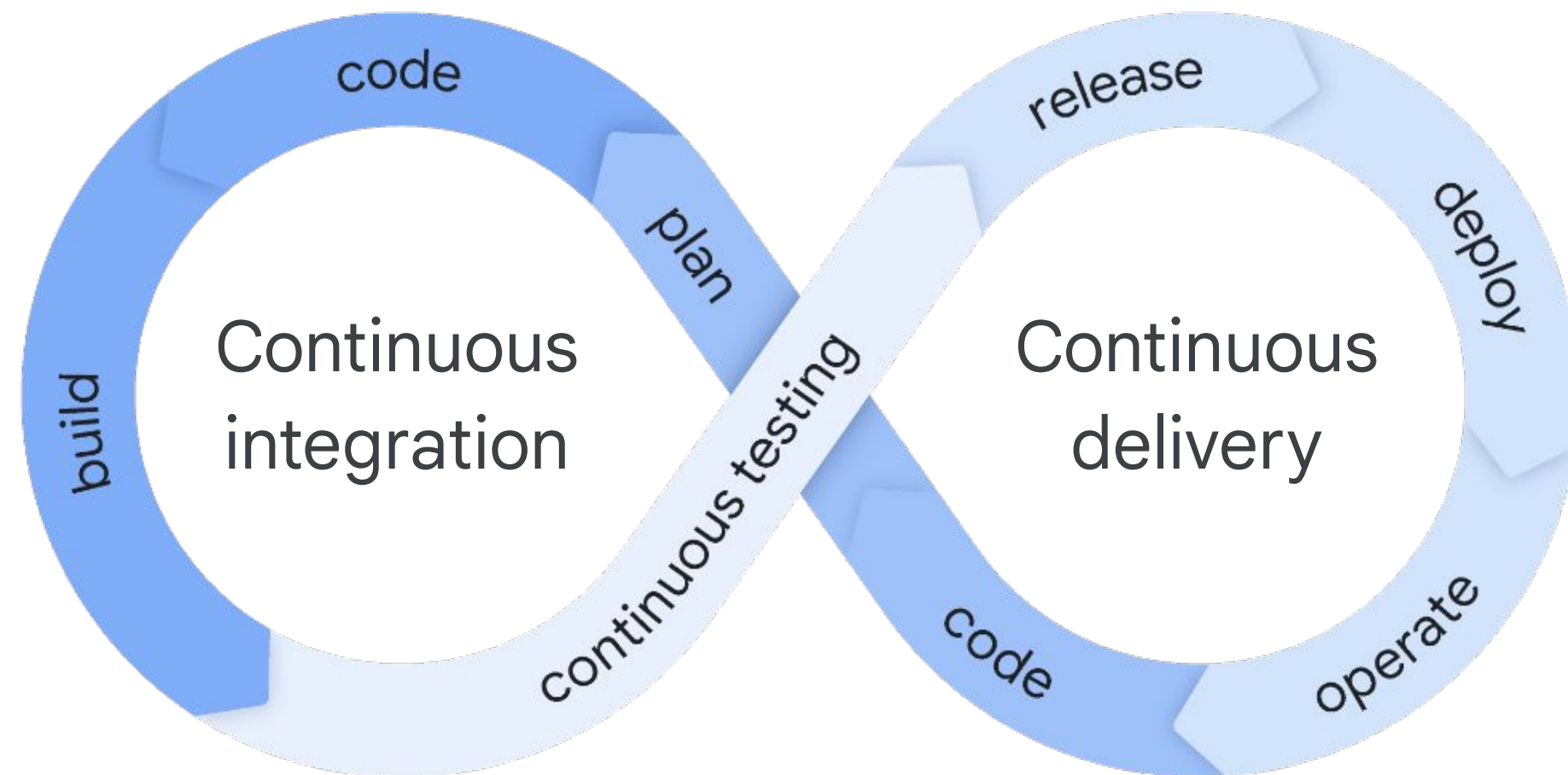
- 01 Continuous integration and continuous deployment (CI/CD)
- 02 Constructing a CI/CD pipeline
- 03 CI/CD products
- 04 Best practices for using CI/CD on Google Cloud



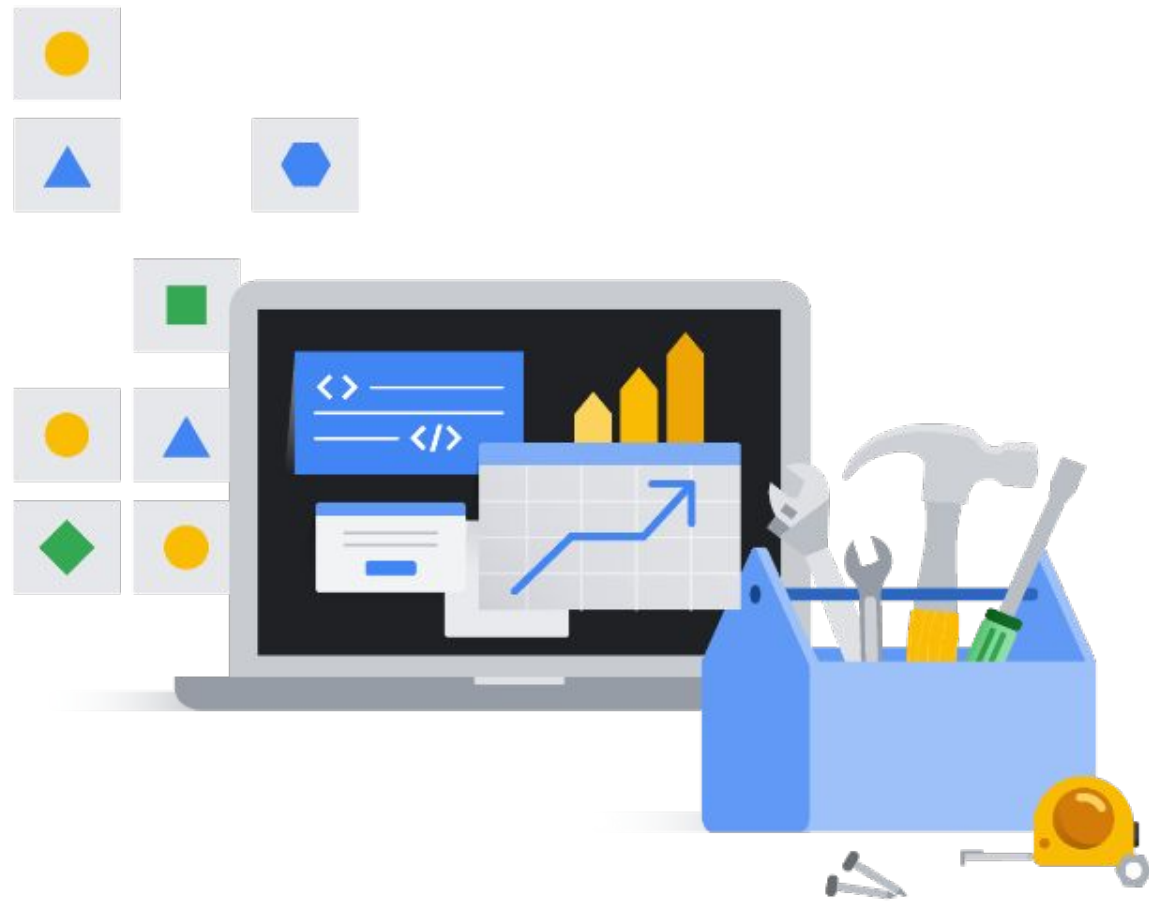
CI/CD:

Continuous integration and continuous delivery

Software development practice that automates the process of building, testing, and deploying code changes.



Continuous integration

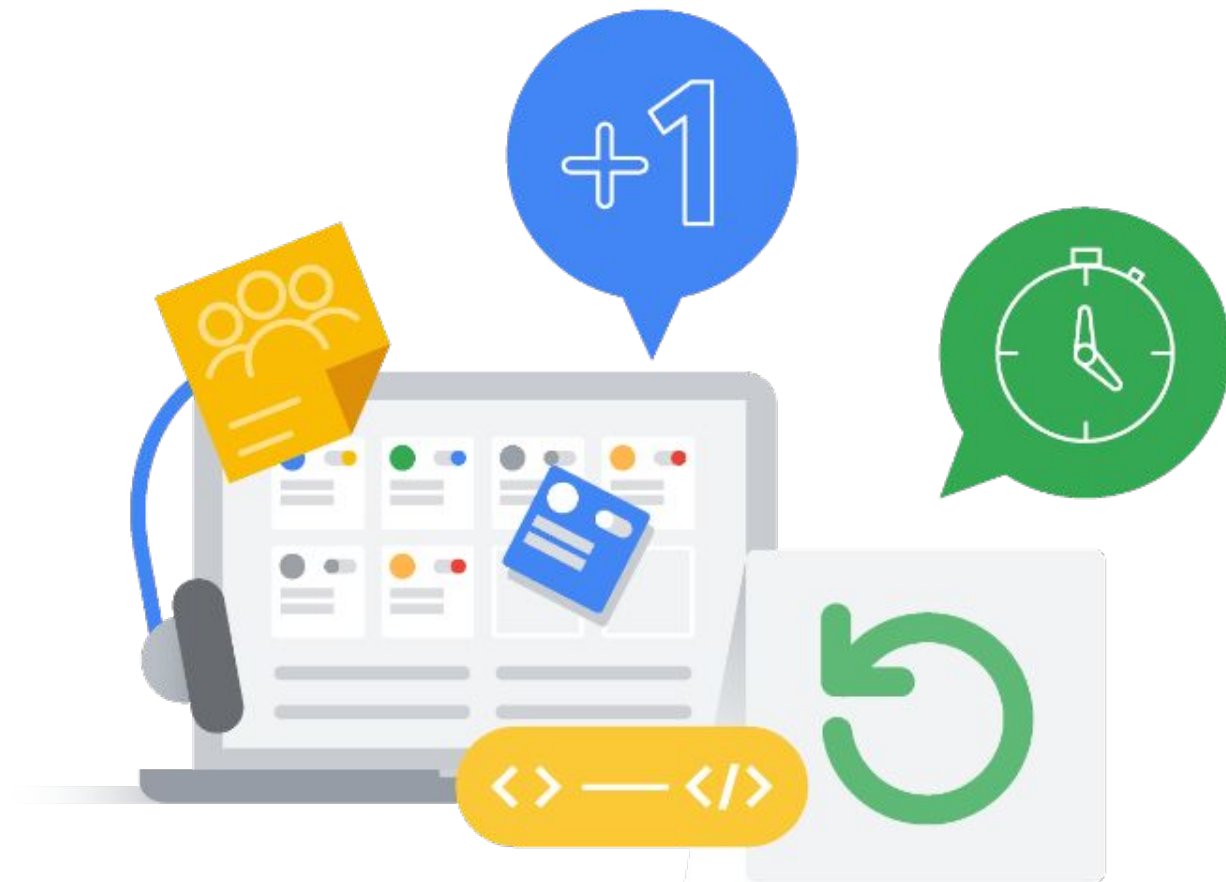


The practice of frequently merging code changes into a central repository.

Repository ensures code remains functional through automated:

- ✓ Builds
- ✓ Tests

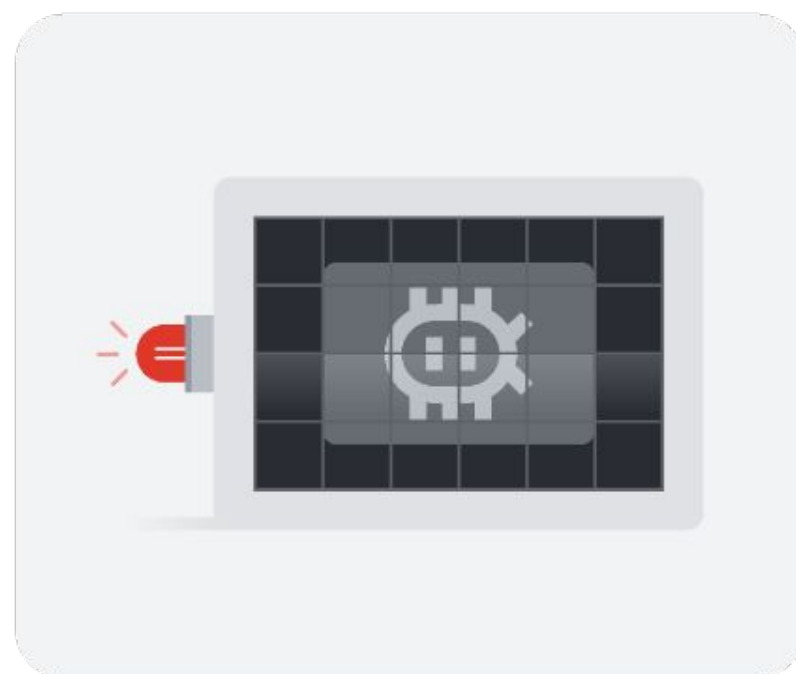
Continuous delivery



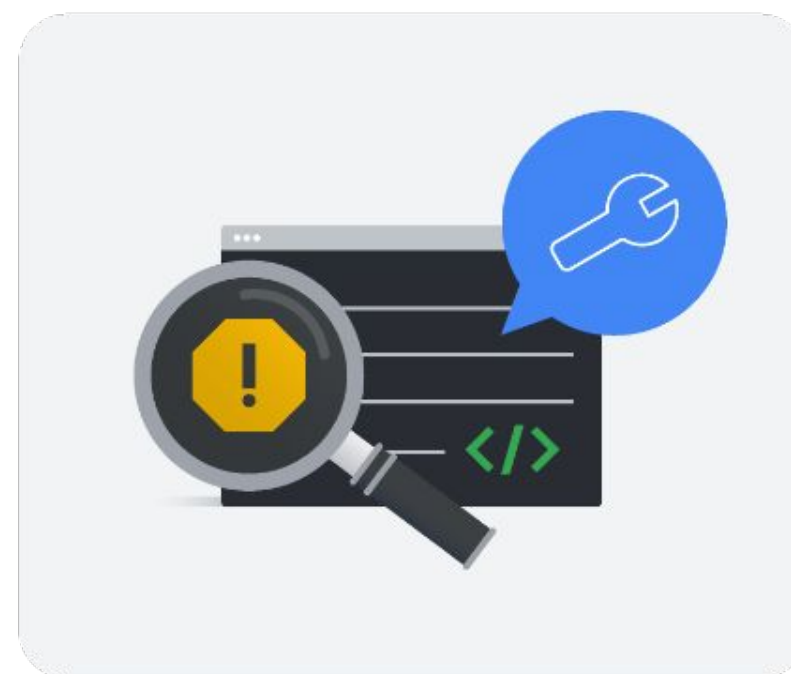
Producing software in short cycles and using processes to ensure a reliable release to production.

- ✓ Helps minimize the risk of loss of service quality.
- ✓ Might mean pushing releases to production several times a day.

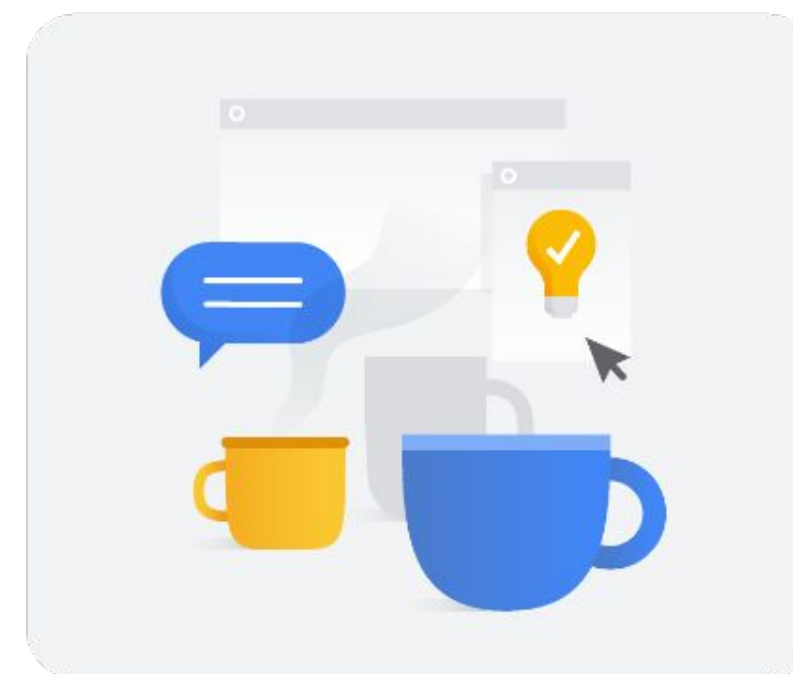
The benefits of CI/CD



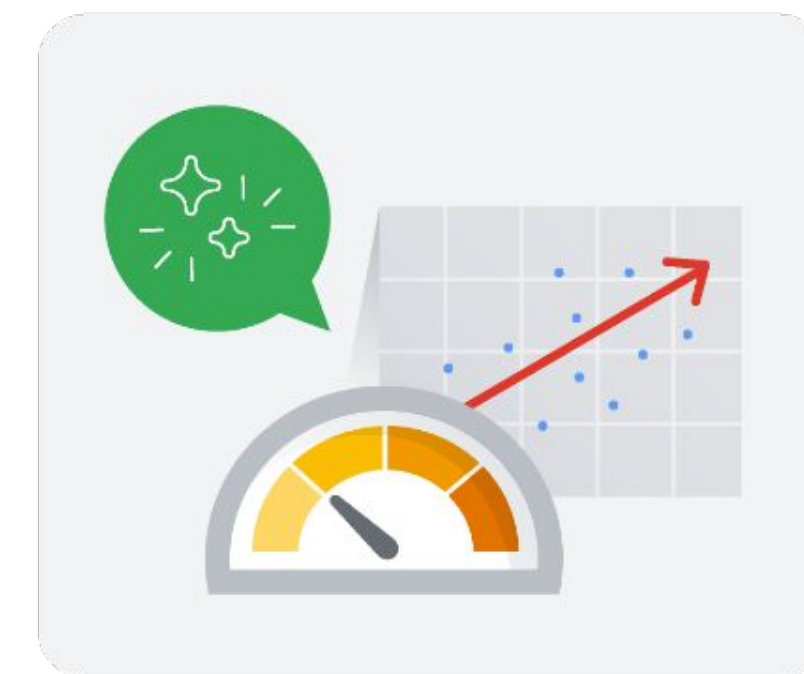
Helps detect bugs sooner.



Helps improve code quality.



Helps enhance collaboration.



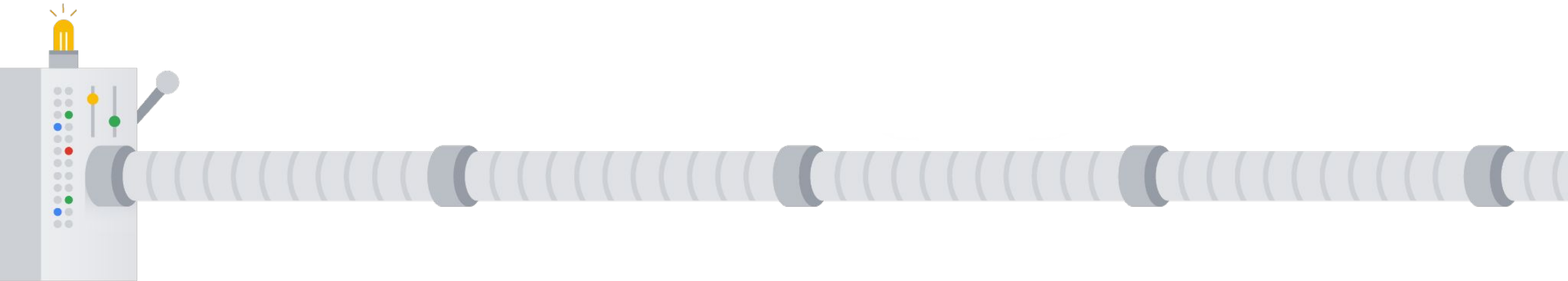
Helps reduce the risk of regressions.

Using CI/CD with Google Kubernetes Engine

- 01 Continuous integration and continuous deployment (CI/CD)
- 02 Constructing a CI/CD pipeline**
- 03 CI/CD products
- 04 Best practices for using CI/CD on Google Cloud

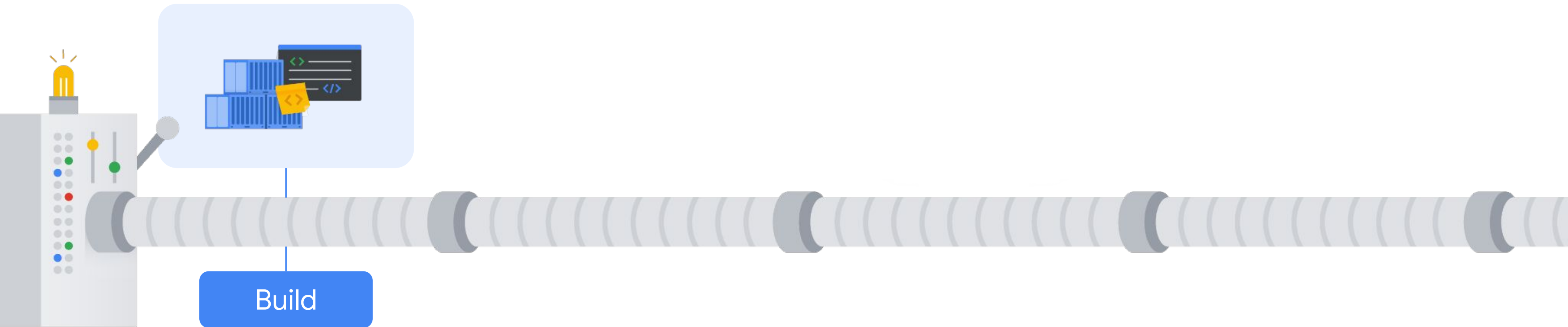


CI/CD pipelines



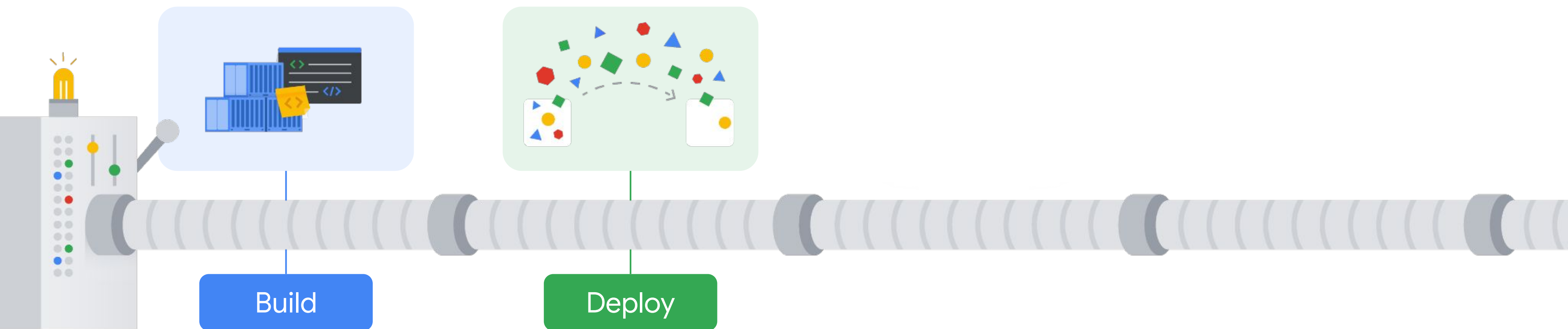
Pipelines can be triggered manually or automatically.

CI/CD pipeline stages



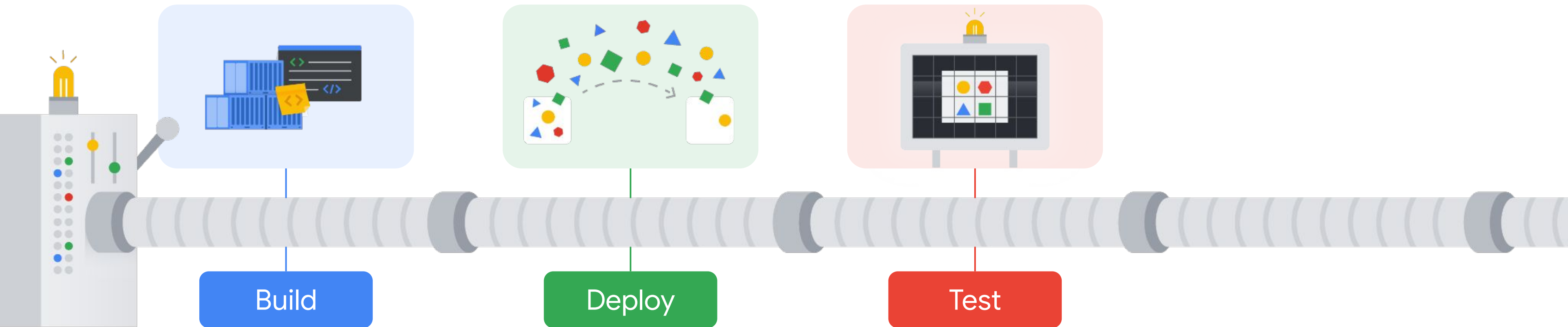
Codebase is checked out to a specific version and artifacts are built.

CI/CD pipeline stages



Artifacts are deployed into a test environment that replicates the production environment.

CI/CD pipeline stages



The application is rigorously tested to ensure its quality is high.

Unit

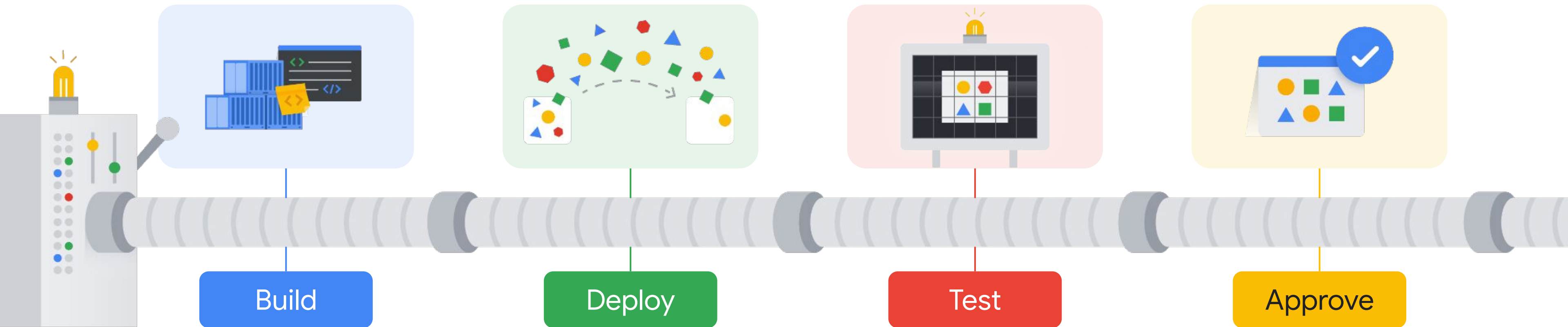
Functional

Integration

Vulnerability

Performance

CI/CD pipeline stages



Developers will decide if a pipeline should proceed to the production environment.

There is no single version of a CI/CD pipeline



Manual pipelines
(no CI/CD)



Packaged tools
with a fixed set of
built-in
automation

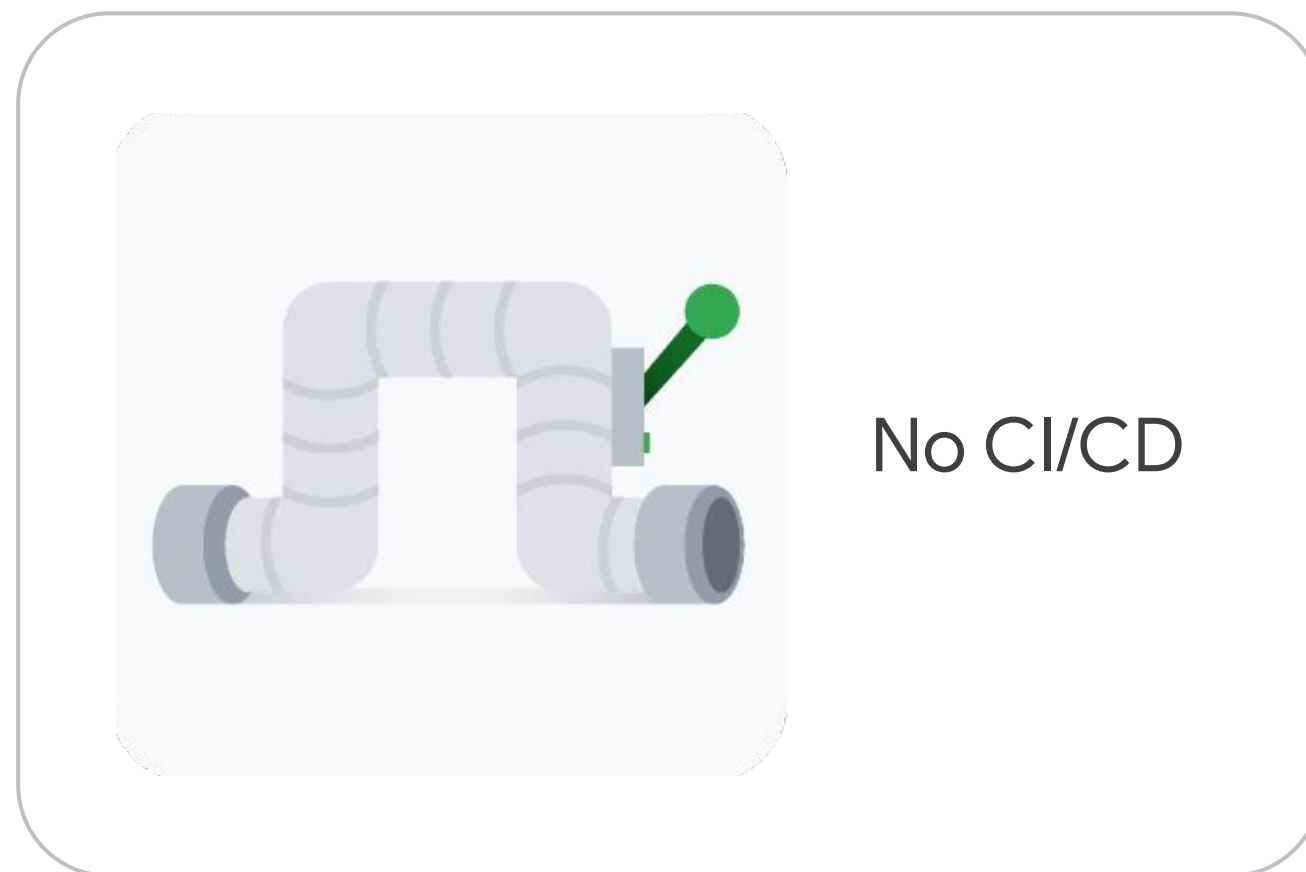


Dev-centric
CI/CD



Ops-centric
CI/CD

Manual pipelines



Each developer is responsible for:

- Integrating code they write.
- Deploying code they write.
- Ensuring new code doesn't conflict with other code.

Packaged tools with a fixed set of built-in automation



A pipeline reliant on a CI/CD tool

- Allows for very specific customization.
- Can make development difficult to maintain and review.
- Not suitable for complex systems.

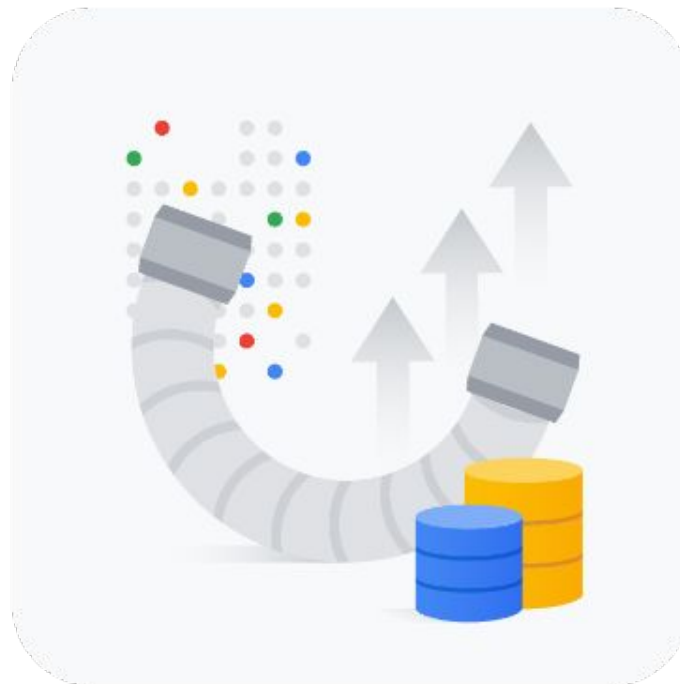
Dev-centric CI/CD



Centers the pipeline around the system's code repository

- GitOps → Stores both the code and configuration files in a source repository, which is used as the authority for an application.
- Use Cloud Build to create dev-centric pipelines.

Ops-centric CI/CD



Prioritizes automating operational tasks to ensure smooth and reliable software delivery.

Common tool: Spinnaker

- Designed to handle very large deployments (thousands).
- Cost can be justified by the scale of the deployment.

Using CI/CD with Google Kubernetes Engine

- 01 Continuous integration and continuous deployment (CI/CD)
- 02 Constructing a CI/CD pipeline
- 03 CI/CD products**
- 04 Best practices for using CI/CD on Google Cloud



CI/CD tools in the Google Cloud Marketplace

Jenkins

One of the oldest open-source continuous integration servers.

Spinnaker

Open-source, multi-cloud, continuous delivery platform for software releases.

CircleCI

A continuous integration and delivery platform for teams of all sizes.

GitLab CI

A continuous integration tool built into GitLab.

Drone

A CI/CD platform built with a container-first architecture. Runs builds with Docker.

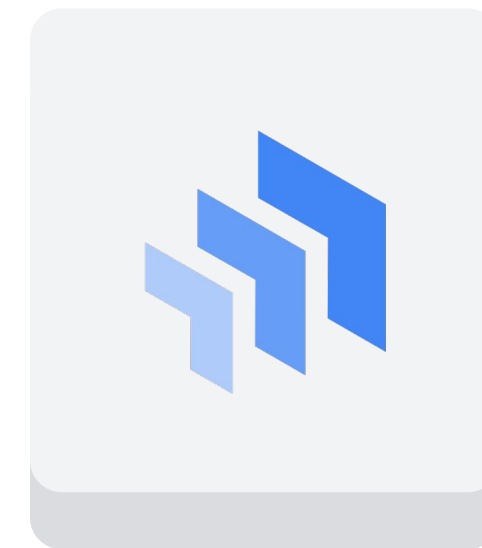
Google Cloud's CI/CD tools



Artifact Registry

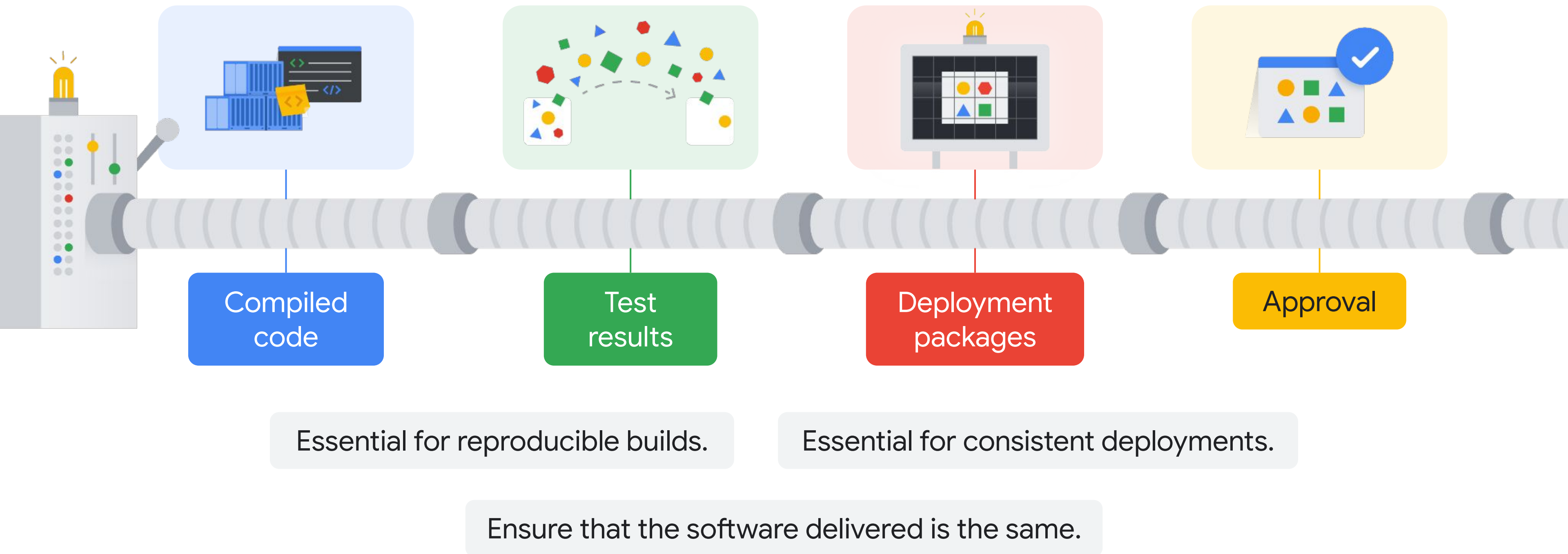


Cloud Build



Cloud Deploy

Artifacts are the outputs produced at each stage



Artifact Registry



Artifact Registry



A fully-managed, secure, and scalable artifact repository.



Stores, manages, and distributes build artifacts.



Can store artifacts of any type, including:



Source code

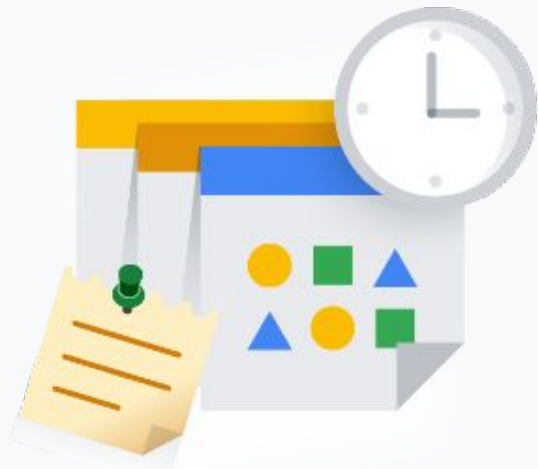


Binaries



Docker images

Artifact Registry features



Version control

Used to track changes over time.

Helpful for:

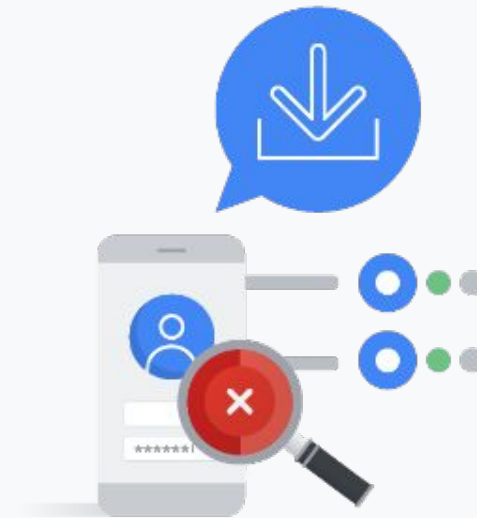
- Debugging
- Rollbacks



Retention policies

A way to retain or delete articles for a period of time.

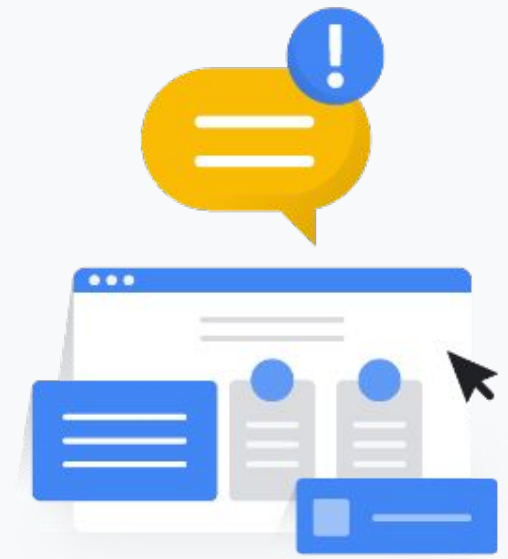
- Reduces the risk of data loss.
- Frees up storage.



Access controls

Define who can view or download artifacts.

- Protects sensitive artifacts.



Webhooks

Automated notifications sent to a specific URL.

- Keeps track of changes.

Other important information about Artifact Registry



Artifact Registry



Built on top of Google Cloud Storage.



Supports a variety of artifact formats, including AAR, APK, JAR, POM, and ZIP.

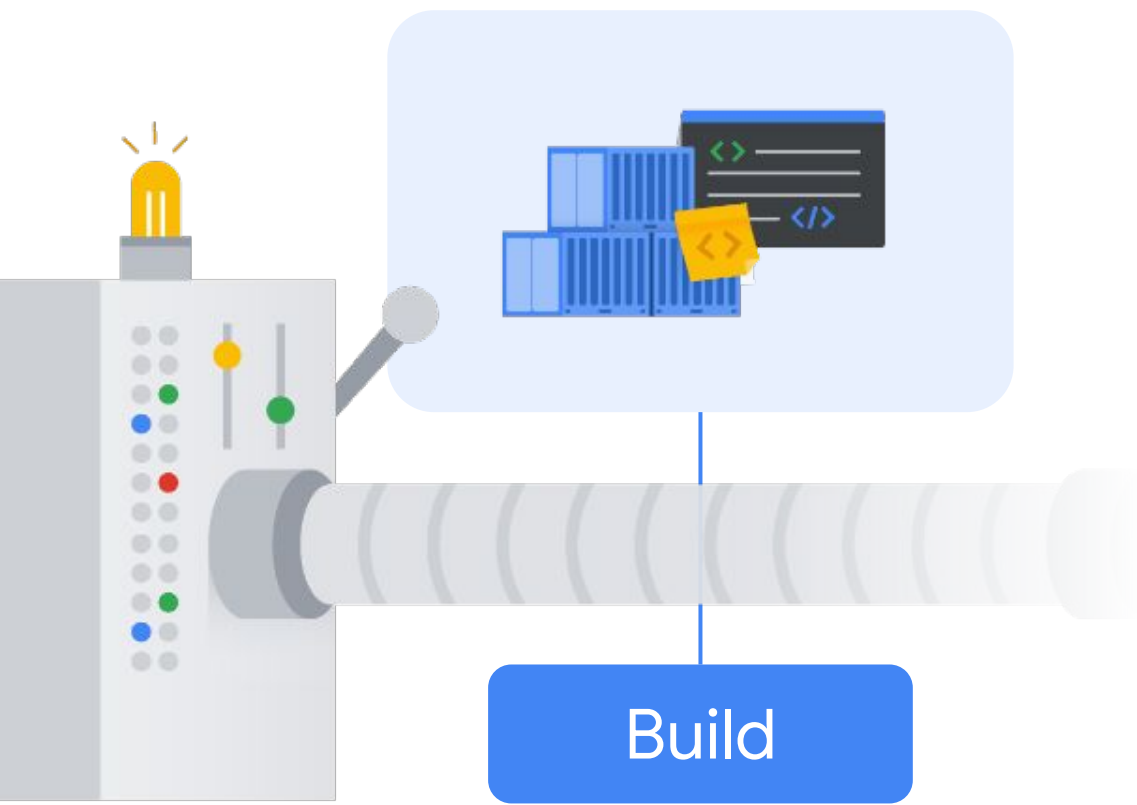


Can be integrated with a variety of build tools, including Gradle, Maven, and Bazel.



Available in all Google Cloud regions.

Cloud Build



When raw source code is transformed into a deployable package.



Cloud Build

Can execute builds on:

- Google Cloud infrastructure.
- An on-premises environment.
- A combination of the two.

Cloud Build



Cloud Build



Can import source code from a variety of repositories or cloud storage space.



Can execute a build to defined specifications.

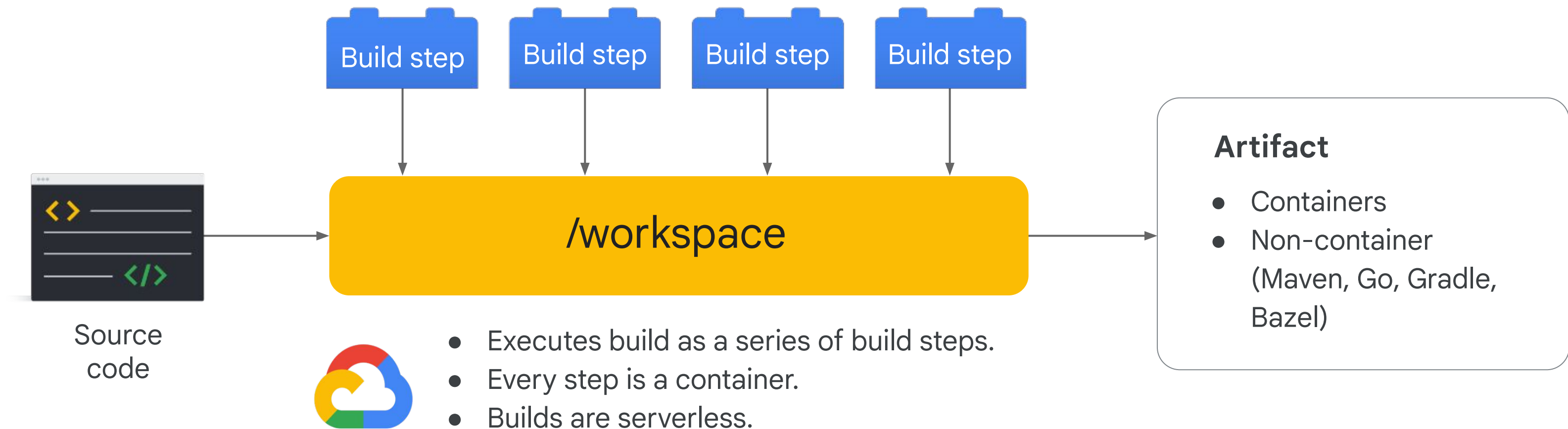


Can produce artifacts, such as container images.

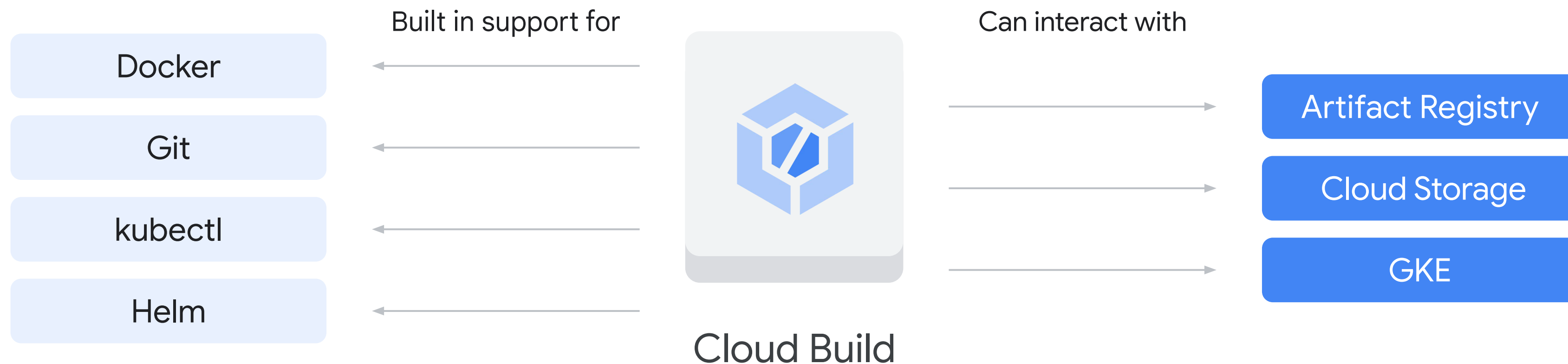


Can automatically start the build process in response to specific changes in your code.

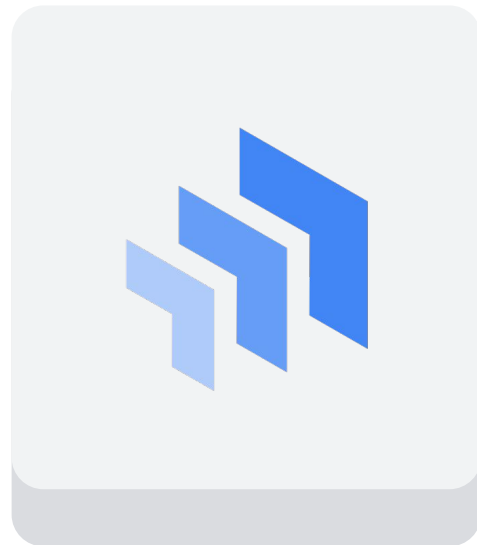
The build process



Cloud Build features



Cloud Deploy



Cloud Deploy



Automates the deployment of applications to GKE.



Handles the continuous delivery (CD) stage.



Connects the build artifacts to the deployment environment.



Provides a single place to view and manage all of your deployments.



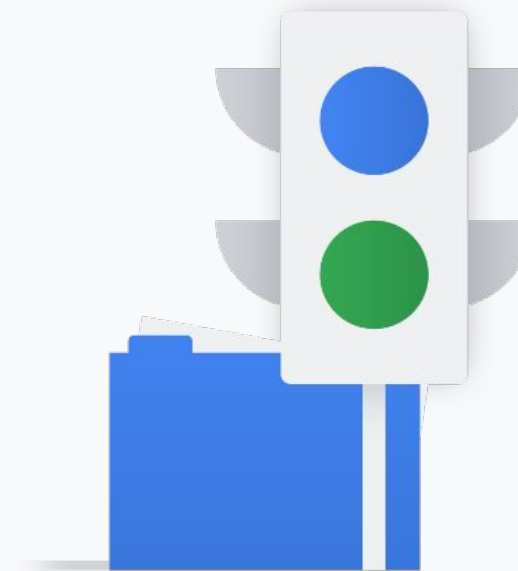
Can be integrated with other Google Cloud services (Cloud Monitoring, Cloud Logging).

Cloud Deploy features



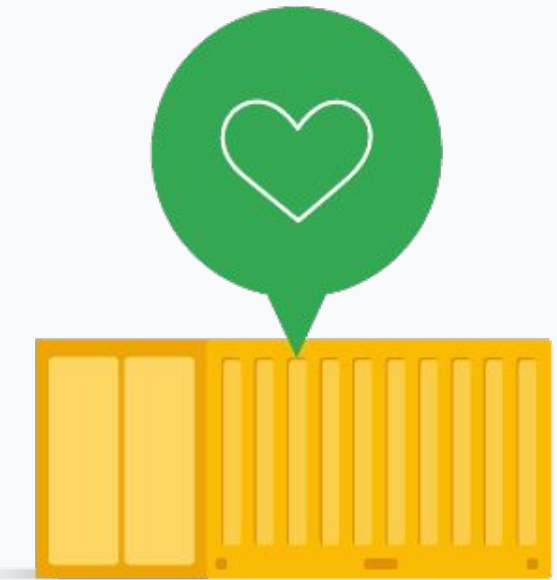
Can deploy apps from many sources:

- Source code repositories (GitHub and Bitbucket)
- Container images (Docker and Google Artifact Registry)
- Helm charts



Supports deployment methods:

- Blue/green deployments
- Rolling updates
- Canary deployments



Can help improve Kubernetes deployments in terms of:

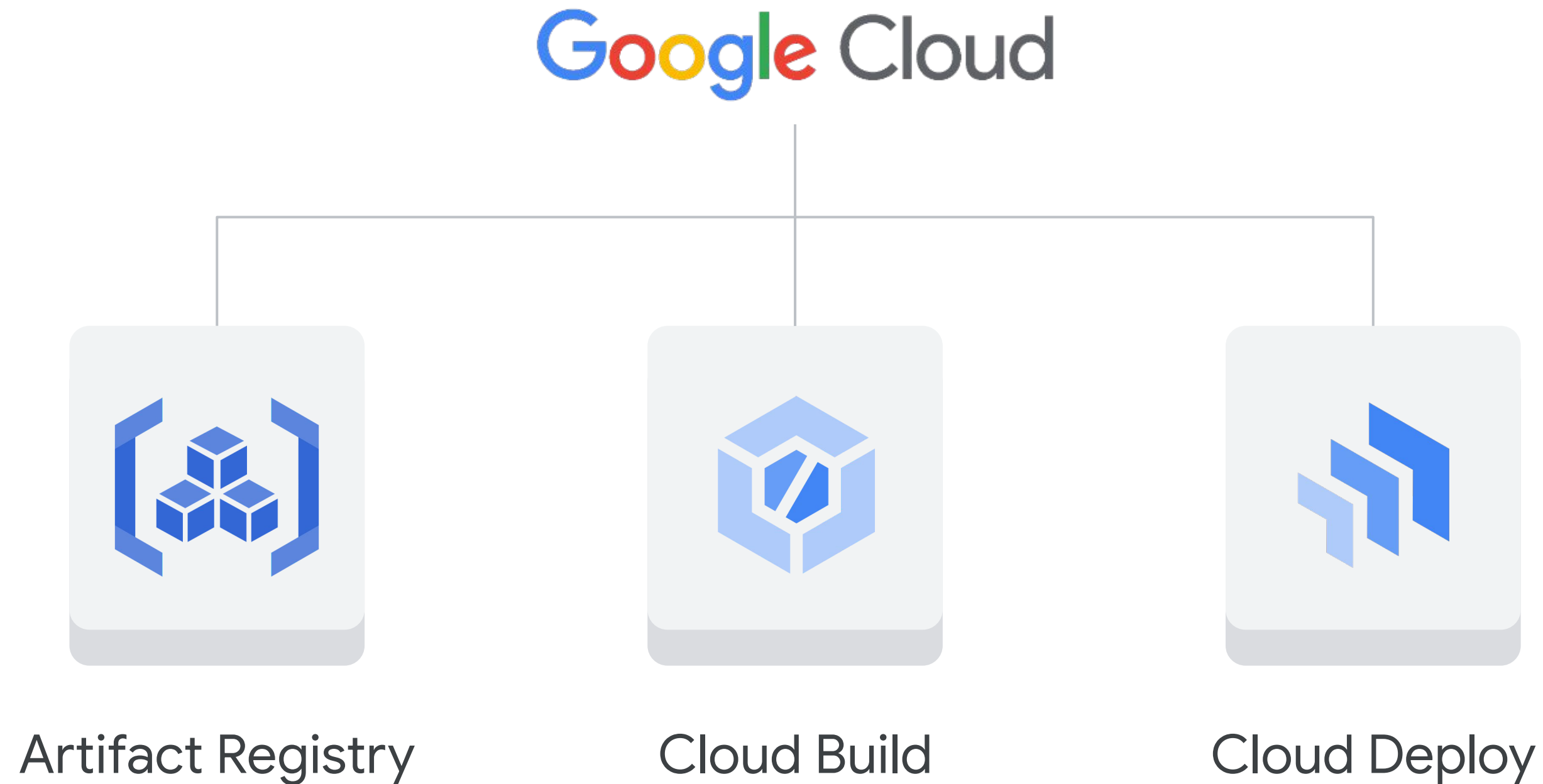
- Reliability
- Scalability
- Security

Using CI/CD with Google Kubernetes Engine

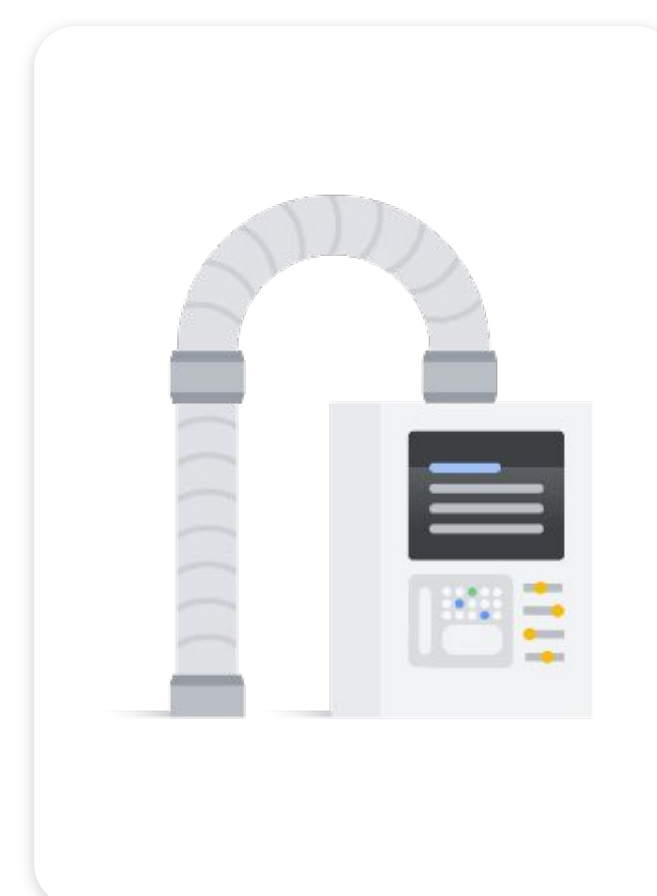
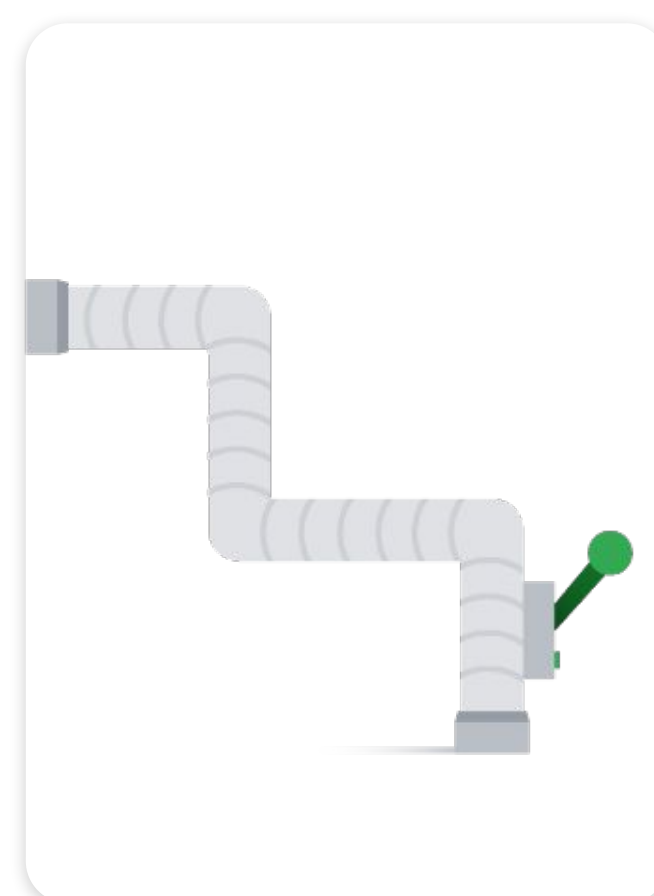
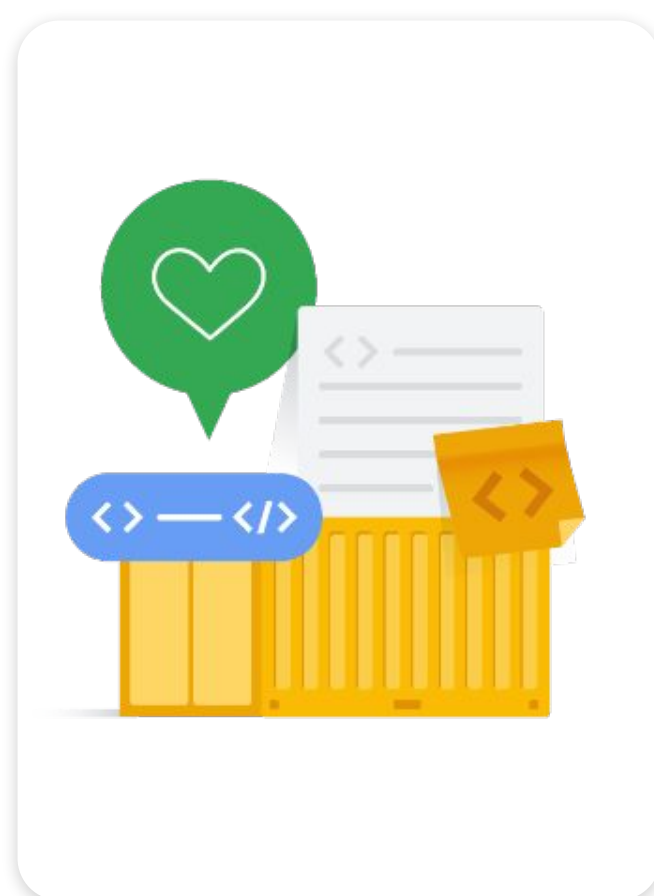
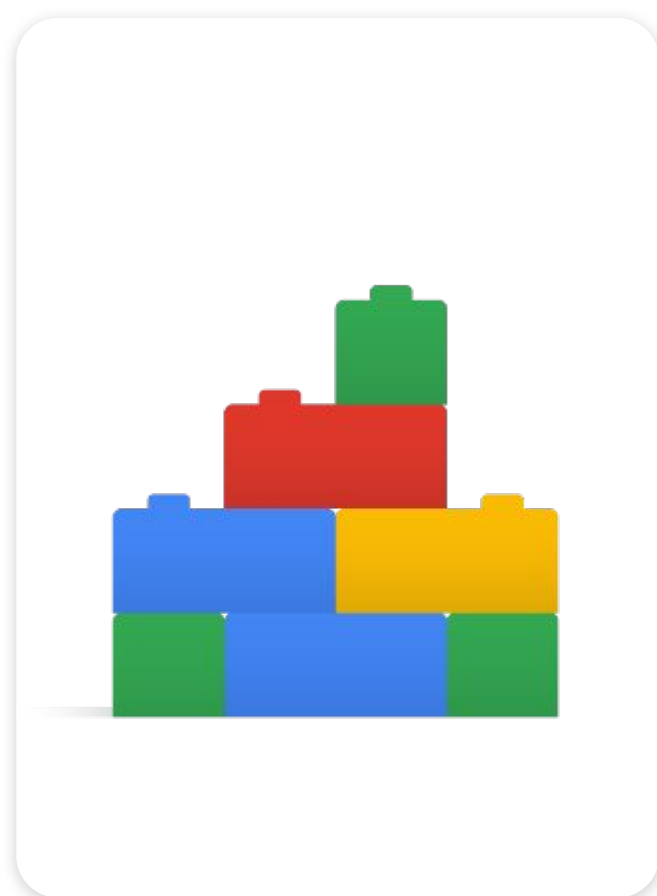
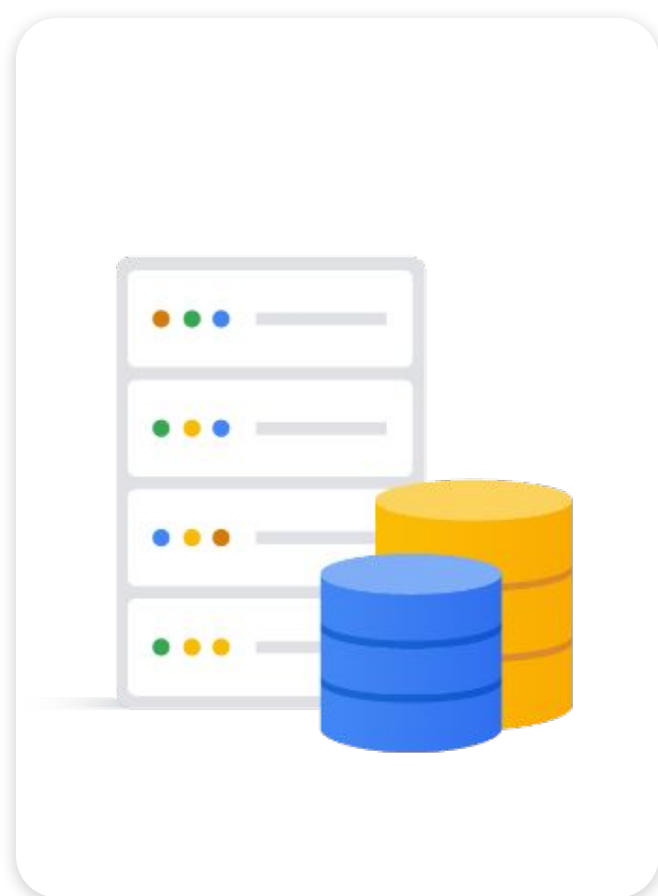
- 01 Continuous integration and continuous deployment (CI/CD)
- 02 Constructing a CI/CD pipeline
- 03 CI/CD products
- 04** Best practices for using CI/CD on Google Cloud



CI/CD enables faster, more reliable software releases

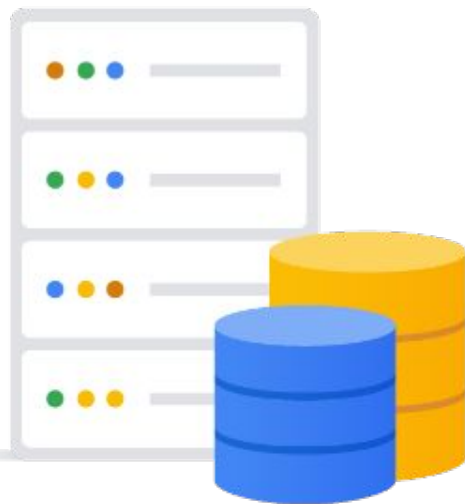


CI/CD best practices



CI/CD best practices

Use a managed artifact repository.



Artifact
Registry

A managed, secure, and scalable artifact repository.

Stores, manages, and distributes build artifacts.

Helps improve the efficiency and reliability of your CI/CD pipeline.

CI/CD best practices

Use a build service.

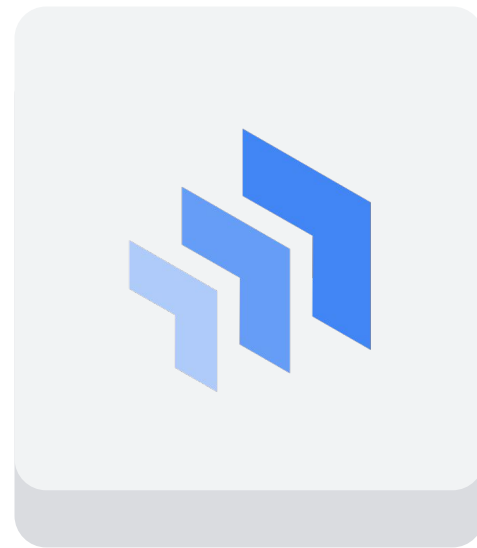


Cloud
Build

Helps to improve the scalability and flexibility of your CI/CD pipeline.

CI/CD best practices

Use a
deployment
service.



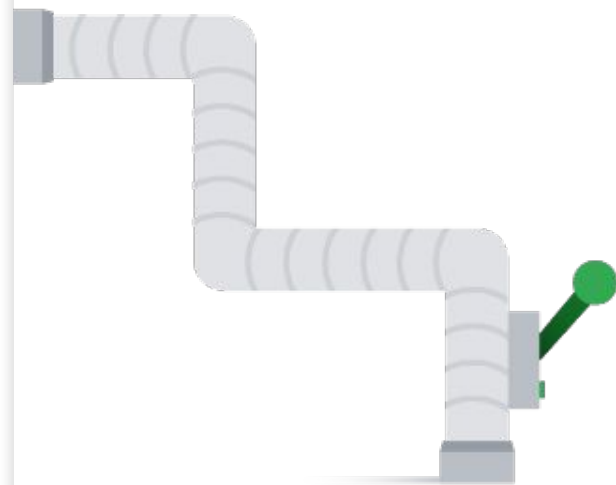
Cloud
Deploy

Helps automate and simplify the
deployment process.

Can help improve the reliability
and scalability of your
applications.

CI/CD best practices

Automate your pipeline.



Can help reduce the risk of human error.

Can help improve the efficiency of your pipeline.

CI/CD best practices

Monitor your pipeline.



Helps identify and troubleshoot any problems.

Can help improve the performance of your pipeline.

CI/CD best practices

Use a managed artifact repository.



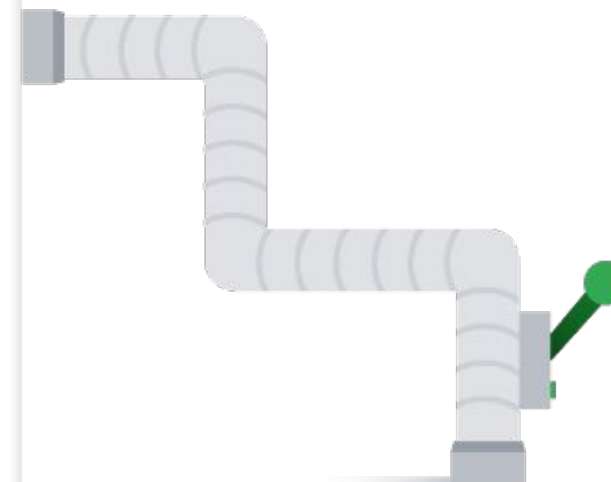
Use a build service.



Use a deployment service.



Automate your pipeline.

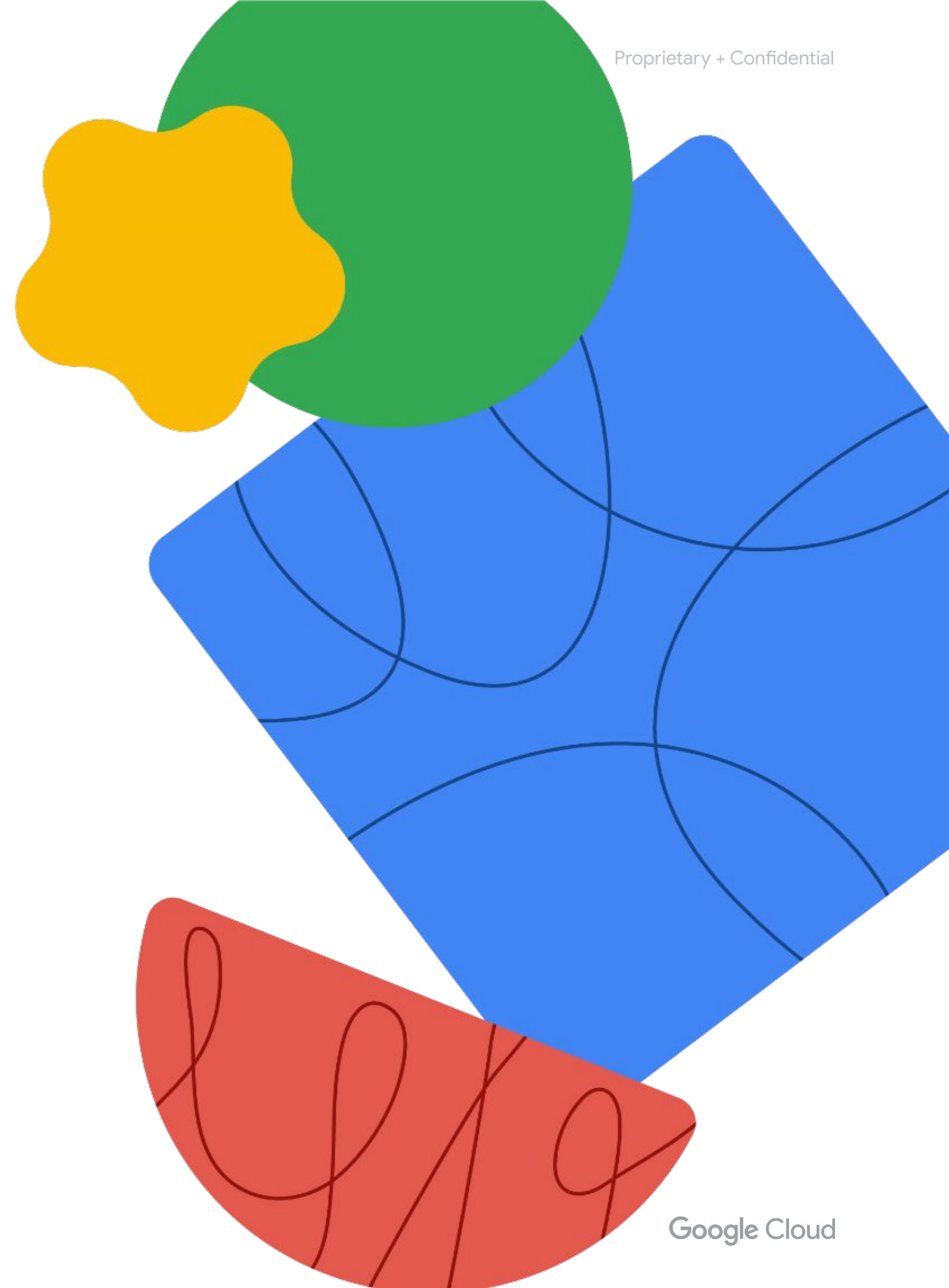


Monitor your pipeline.



Quiz questions

Let's pause for a quick check in.



Quiz | Question 1

Question

Which of the following is not a benefit of CI/CD?

- A. Increased speed of software delivery.
- B. Improved quality of software.
- C. Reduced risk of breaking existing functionalities.
- D. More efficient use of resources.

Quiz | Question 1

Answer

Which of the following is not a benefit of CI/CD?

- A. Increased speed of software delivery.
- B. Improved quality of software.
- C. Reduced risk of breaking existing functionalities.
- D. More efficient use of resources.



Quiz | Question 2

Question

CI/CD pipelines exist along a spectrum of implementations. Google Cloud tools such as Cloud Build create pipelines at which level of the spectrum?

- A. Manual
- B. Built-in automation
- C. Dev-centric CI/CD
- D. Ops-centric CI/CD

Quiz | Question 2

Answer

CI/CD pipelines exist along a spectrum of implementations. Google Cloud tools such as Cloud Build create pipelines at which level of the spectrum?

- A. Manual
- B. Built-in automation
- C. Dev-centric CI/CD
- D. Ops-centric CI/CD



Quiz | Question 3

Question

Which of the following is **not** a responsibility of Cloud Deploy?

- A. Configuring CI/CD pipelines.
- B. Ensuring that applications are up-to-date.
- C. Rolling back deployments.
- D. Deploying applications to Kubernetes clusters.

Quiz | Question 3

Answer

Which of the following is **not** a responsibility of Cloud Deploy?

- A. Configuring CI/CD pipelines.
- B. Ensuring that applications are up-to-date.
- C. Rolling back deployments.
- D. Deploying applications to Kubernetes clusters.



Quiz | Question 4

Question

Which of the following is **not** a recommended practice for a GKE CI/CD pipeline?

- A. Using a version control system like Git to track changes.
- B. Manually testing the application after each build.
- C. Ensuring that the pipeline is automated as much as possible.
- D. Deploying the application to a staging environment before production.

Quiz | Question 4

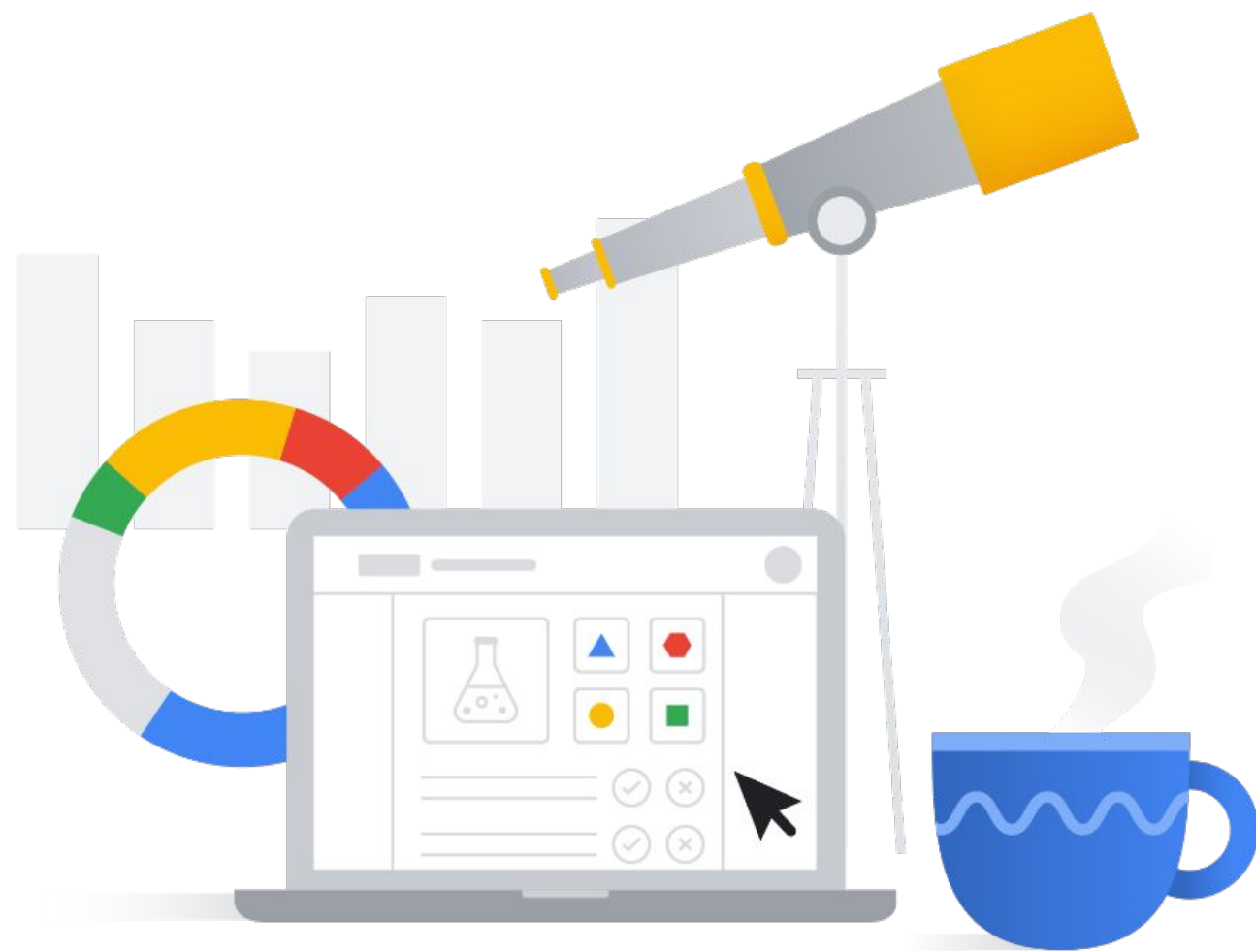
Answer

Which of the following is **not** a recommended practice for a GKE CI/CD pipeline?

- A. Using a version control system like Git to track changes.
- B. Manually testing the application after each build.
- C. Ensuring that the pipeline is automated as much as possible.
- D. Deploying the application to a staging environment before production.



Lab: Continuous Delivery with Google Cloud Deploy

**01**

Deploy a container image to Google Cloud Artifact Registry using Scaffold

02

Create a Google Cloud Deploy delivery pipeline

03

Create a release for the delivery pipeline

04

Promote the application through the targets in the delivery pipeline.